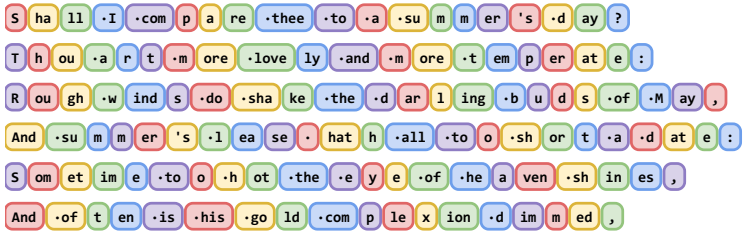
UNIT 01 · CHAPTER 5

BPE Tokenization

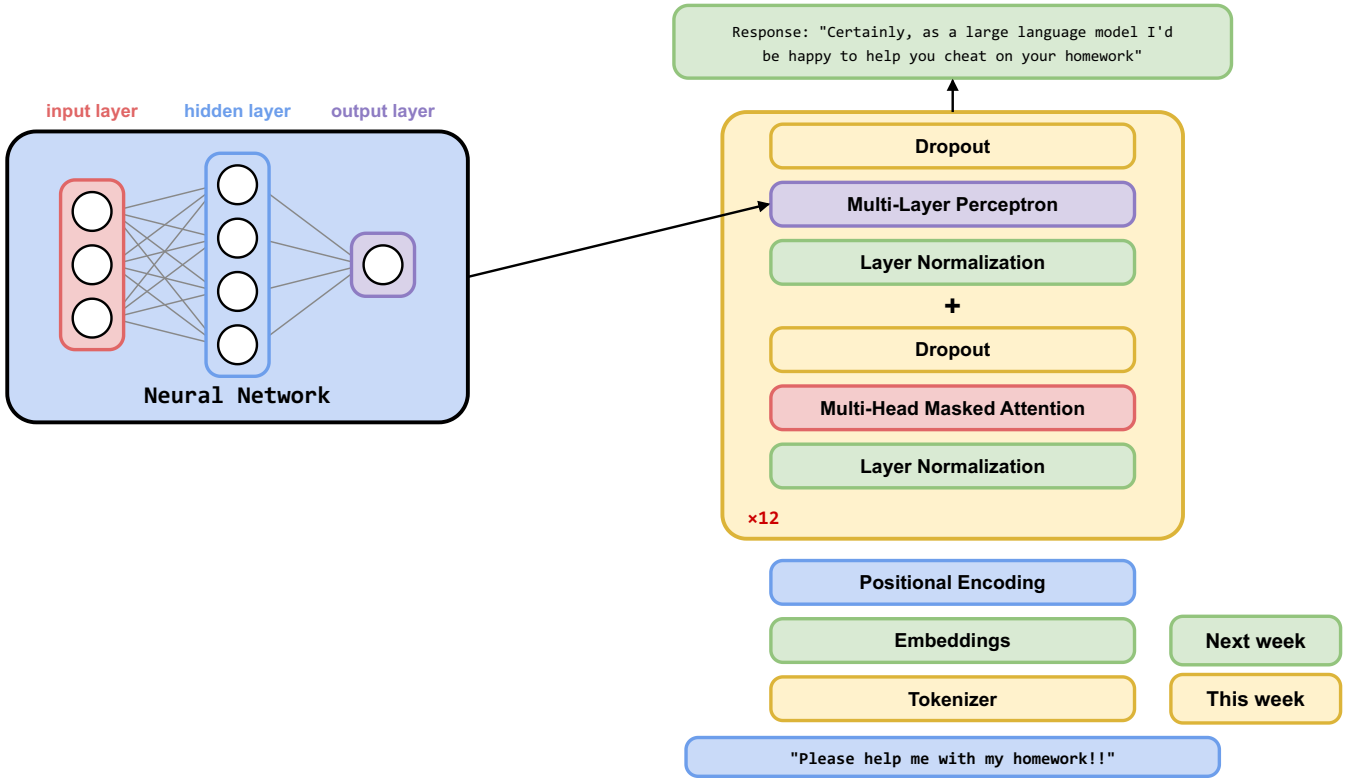


CSE 10124 · Building ChatGPT

William Theisen · University of Notre Dame

CSE 10124: Building ChatGPT

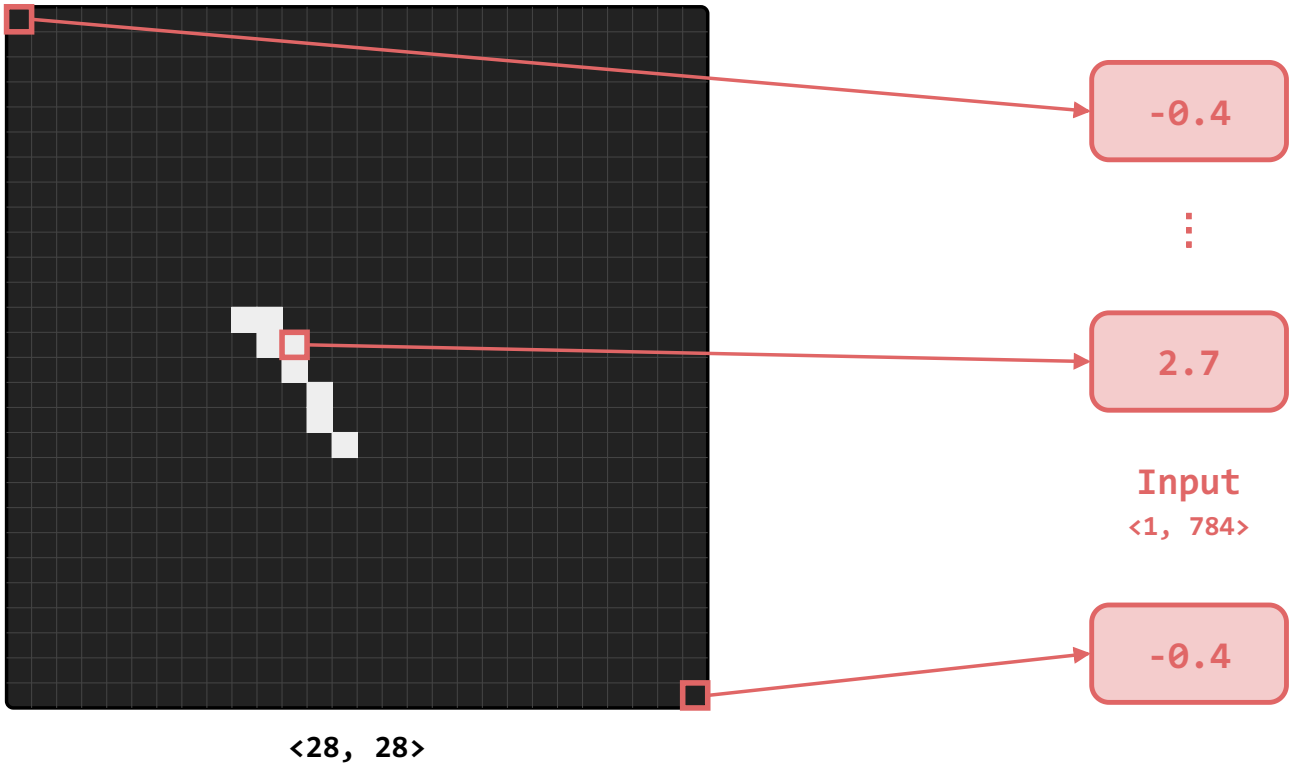
Last week we built a simple **neural network**. This is one of the pieces we need to build **GPT-2**.



Over the next two weeks, we'll build the pieces we need to actually get **input** into the model.

Representing Image Input

Our images are <28, 28> so we can **flatten** them into a <1, 784> dimensional **vector**.



Different Tokenization Levels

Input Prompt

"The cat sat on the mat and ate."

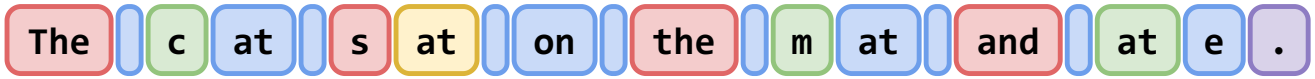
◆ **Word Level**



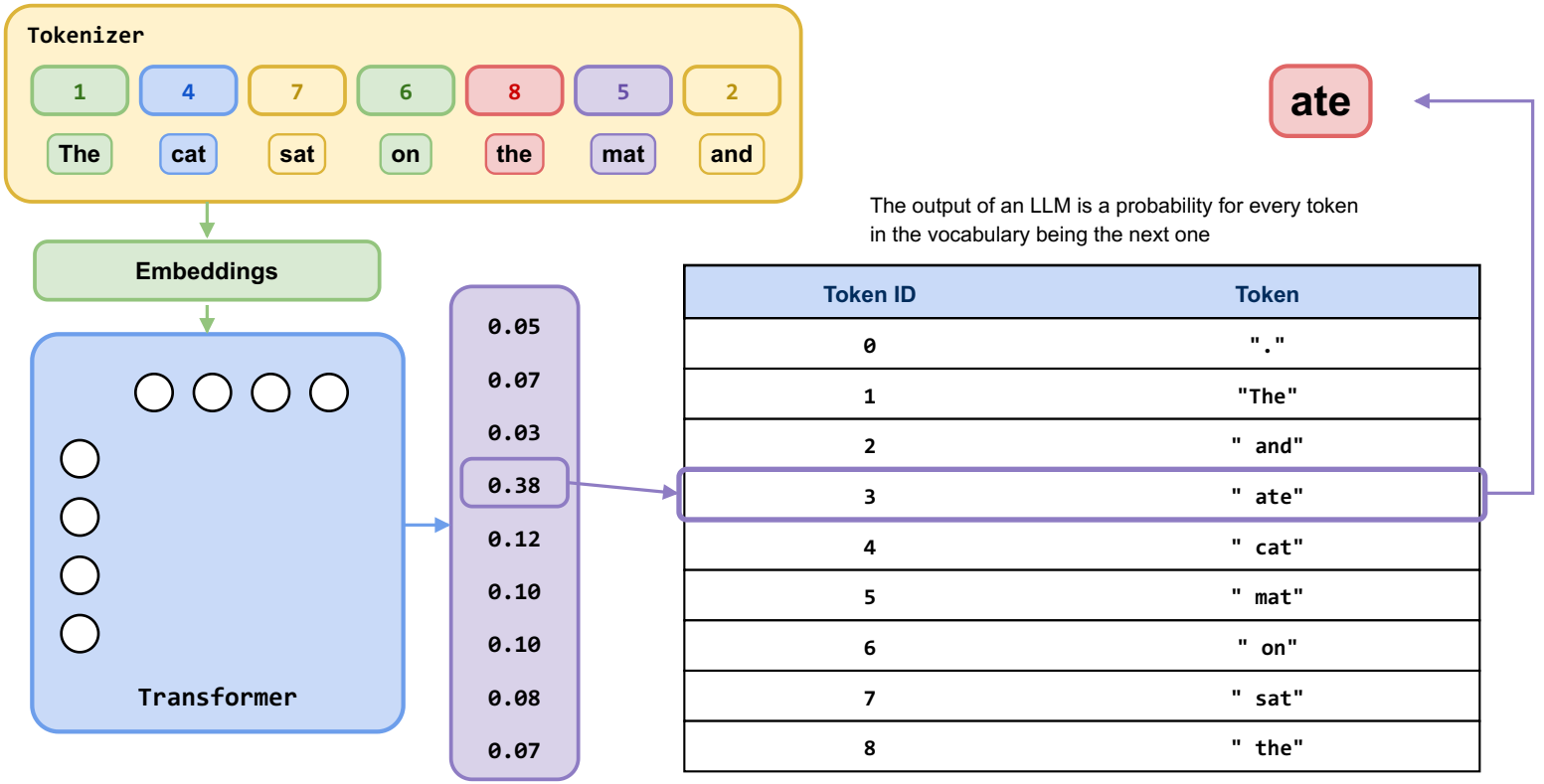
◆ **Character Level**



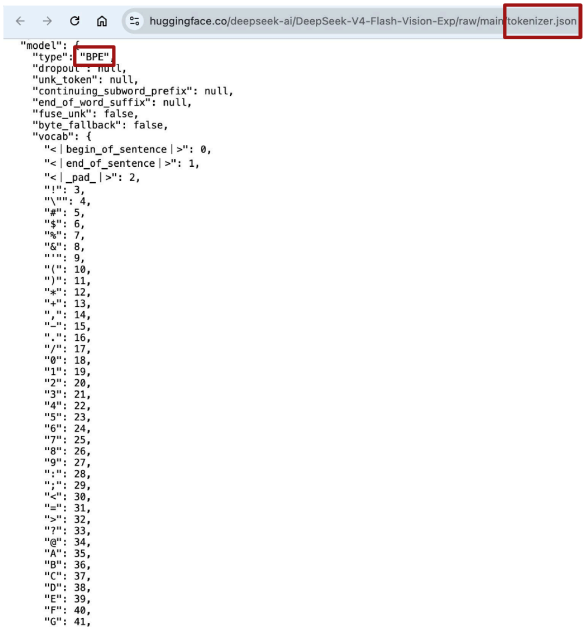
◆ **Subword** : a reusable piece smaller than a whole word



Output of an LLM



Tokenization in LLMs



JSON: **J**ava**S**cript **O**bject **N**otation

Byte-Pair Encoding Tokenization

```
1  print(list('hello there 金'.encode('utf-8')))
```

OUTPUT

```
[104, 101, 108, 108, 111, 32, 116, 104, 101, 114,
```

```
1  data = 'hello there 金'.encode('utf-8')
2  print([bin(num) for num in list(data)])
```

OUTPUT

```
['0b1101000', '0b1100101', '0b1101100', '0b110110(
```

By **tokenizing** at the **byte** level we could theoretically process any kind of text that could be passed in as a prompt.

programming: ASCII Values

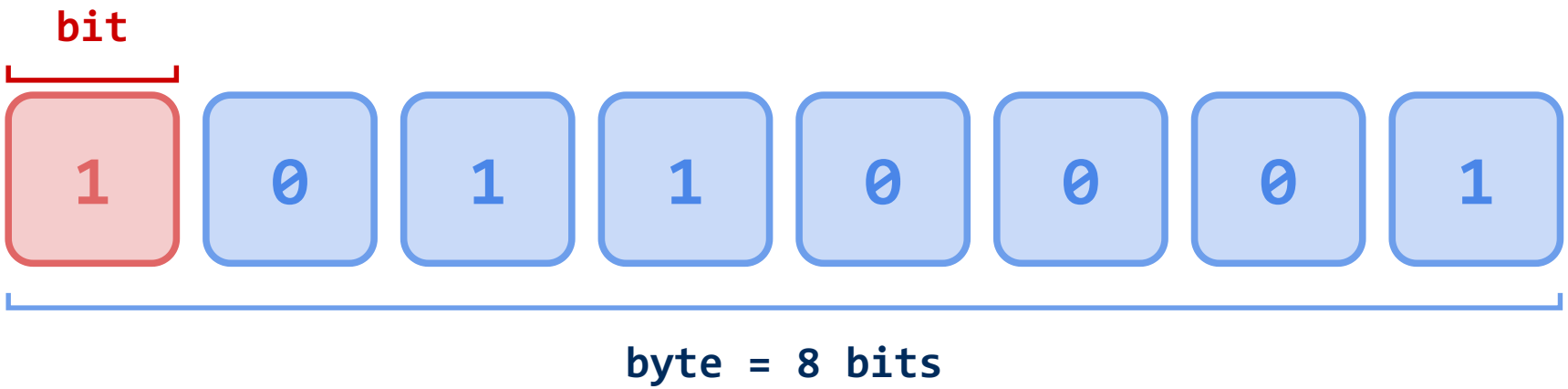
DEC	BIN	SYM	DEC	BIN	SYM	DEC	BIN	SYM
32	00100000	SP	48	00110000	0	64	01000000	@
33	00100001	!	49	00110001	1	65	01000001	A
34	00100010	"	50	00110010	2	66	01000010	B
35	00100011	#	51	00110011	3	67	01000011	C
36	00100100	$	52	00110100	4	68	01000100	D
37	00100101	%	53	00110101	5	69	01000101	E
38	00100110	&	54	00110110	6	70	01000110	F
39	00100111	'	55	00110111	7	71	01000111	G
40	00101000	(	56	00111000	8	72	01001000	H
41	00101001	)	57	00111001	9	73	01001001	I
42	00101010	*	58	00111010	:	74	01001010	J
43	00101011	+	59	00111011	;	75	01001011	K
44	00101100	,	60	00111100	<	76	01001100	L
45	00101101	-	61	00111101	=	77	01001101	M
46	00101110	.	62	00111110	>	78	01001110	N
47	00101111	/	63	00111111	?	79	01001111	O

programming: ASCII Values

DEC	BIN	SYM	DEC	BIN	SYM	DEC	BIN	SYM
80	01010000	P	96	01100000	`	112	01110000	p
81	01010001	Q	97	01100001	a	113	01110001	q
82	01010010	R	98	01100010	b	114	01110010	r
83	01010011	S	99	01100011	c	115	01110011	s
84	01010100	T	100	01100100	d	116	01110100	t
85	01010101	U	101	01100101	e	117	01110101	u
86	01010110	V	102	01100110	f	118	01110110	v
87	01010111	W	103	01100111	g	119	01110111	w
88	01011000	X	104	01101000	h	120	01111000	x
89	01011001	Y	105	01101001	i	121	01111001	y
90	01011010	Z	106	01101010	j	122	01111010	z
91	01011011	[	107	01101011	k	123	01111011	{
92	01011100	\	108	01101100	l	124	01111100	|
93	01011101	]	109	01101101	m	125	01111101	}
94	01011110	^	110	01101110	n	126	01111110	~
95	01011111	_	111	01101111	o	127	01111111	DEL

`programming`: Bytes

A **byte** is 8 **bits**, which are the 0s and 1s the computer actually "sees".



NOTE: Python shows each **byte** as an `int` from 0 to 255.

Character-Level Token IDs

Input Prompt: `"the cat sat on the mat and ate."`

Token	Token ID	Token	Token ID	Token	Token ID	Token	Token ID
" "	0	"g"	8	"o"	16	"w"	24
"."	1	"h"	9	"p"	17	"x"	25
"a"	2	"i"	10	"q"	18	"y"	26
"b"	3	"j"	11	"r"	19	"z"	27
"c"	4	"k"	12	"s"	20		
"d"	5	"l"	13	"t"	21		
"e"	6	"m"	14	"u"	22		
"f"	7	"n"	15	"v"	23		



`[21, 9, 6, 0, 4, 2, 21, 0, 20, 2, 21, 0, 16, 15, 0, 21, 9, 6, 0, 14, 2, 21, 0, 2, 15, 5, 0, 2, 21, 6, 1]`

Training a BPE Tokenizer

Broadly speaking, we can train our **tokenizer** by **repeatedly looking for common pairings of tokens** and **merging them to create a new token**.

An untrained tokenizer is effectively just a **character-level** tokenizer.




```
[2, 21]: 4    [21, 9]: 2    [9, 6]: 2    [4, 2]: 1    [20, 2]: 1    [16,
15]: 1    [14, 2]: 1    [2, 15]: 1    [15, 5]: 1    [21, 6]: 1
```

`[2, 21]` appears **4** times so we make a new **token**: **at** = **28**

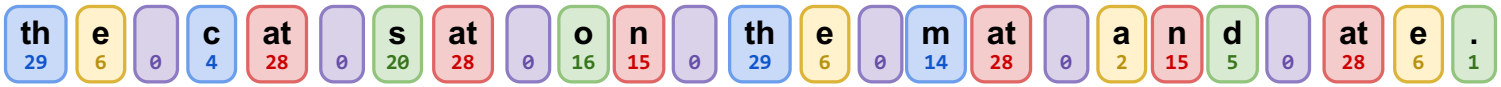


Training a BPE Tokenizer

```
[21, 9]: 2    [9, 6]: 2    [4, 28]: 1    [20, 28]: 1    [16, 15]: 1
[14, 28]: 1    [2, 15]: 1    [15, 5]: 1    [28, 6]: 1
```

`[21, 9]` appears **2** times so we could make a new token: **th** = **29**

Token	Token ID	Token	Token ID	Token	Token ID	Token	Token ID
" "	0	"g"	8	"o"	16	"w"	24
"."	1	"h"	9	"p"	17	"x"	25
"a"	2	"i"	10	"q"	18	"y"	26
"b"	3	"j"	11	"r"	19	"z"	27
"c"	4	"k"	12	"s"	20	"at"	28
"d"	5	"l"	13	"t"	21	"th"	29
"e"	6	"m"	14	"u"	22		
"f"	7	"n"	15	"v"	23		




```
[29, 6, 0, 4, 28, 0, 20, 28, 0, 16, 15, 0, 29, 6, 0, 14, 28, 0, 2, 15, 5,
0, 28, 6, 1]
```

Training a BPE Tokenizer

```
[29, 6]: 2    [4, 28]: 1    [20, 28]: 1    [16, 15]: 1    [14, 28]: 1
[2, 15]: 1    [15, 5]: 1    [28, 6]: 1
```

[29, 6] appears **2** times so we could make a new token: **the** = **30**

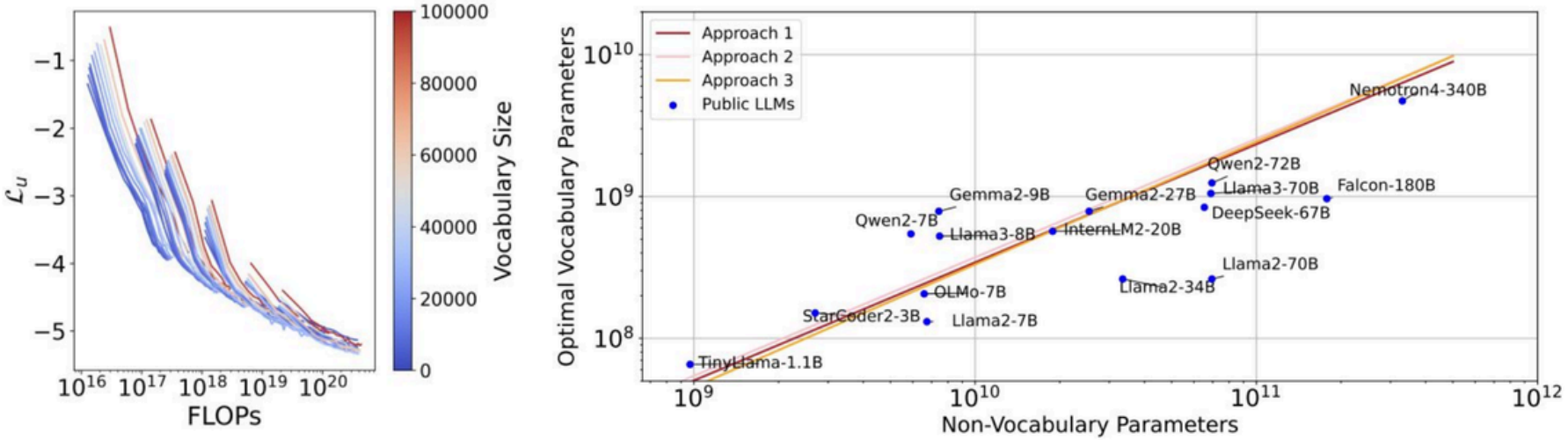
Token	Token ID	Token	Token ID	Token	Token ID	Token	Token ID
" "	0	"g"	8	"o"	16	"w"	24
"."	1	"h"	9	"p"	17	"x"	25
"a"	2	"i"	10	"q"	18	"y"	26
"b"	3	"j"	11	"r"	19	"z"	27
"c"	4	"k"	12	"s"	20	"at"	28
"d"	5	"l"	13	"t"	21	"th"	29
"e"	6	"m"	14	"u"	22	"the"	30
"f"	7	"n"	15	"v"	23		




```
[30, 0, 4, 28, 0, 20, 28, 0, 16, 15, 0, 30, 0, 14, 28, 0, 2, 15, 5, 0, 28,
6, 1]
```

Representative Vocabulary Sizes

- Llama 3: ~128,000 tokens
- Qwen3: ~151,000 tokens
- Gemma 3: ~262,000 tokens



Scaling Laws with Vocabulary:
Larger Models Deserve Larger Vocabularies

Tao et al. (2024), Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies

Programming Day 03: GPT4 Tokenizer

CSE 10124

BPE Tokenization Steps

1.) Convert input to bytes

```
text = 'rug pug hug pun bun hugs run gun'
string_bytes = list(text.encode('utf-8'))
print(string_bytes)
```

OUTPUT

```
[114, 117, 103, 32, 112, 117, 103, 32, 104, 117, 103, 32, 112,
```

2.) Find and count pairs

```
from collections import Counter
def count_pairs(ids):
    return Counter(zip(ids, ids[1:]))

counts = count_pairs(string_bytes)
print(dict(counts))
```

OUTPUT

```
{(114, 117): 2, (117, 103): 4, (103, 32): 3, (32, 112): 2, (11
```

BPE Tokenization Steps

3.) Replace the most common pair with a new token

```python
def replace_pairs(ids, pair, new_id):
    out, i = [], 0
    while i < len(ids):
        if tuple(ids[i:i + 2]) == pair:
            out.append(new_id)
            i += 2
        else:
            out.append(ids[i])
            i += 1
    return out

pair = counts.most_common()[0][0]
merged_string = replace_pairs(string_bytes, pair, 256)
print(merged_string)
```

OUTPUT

```
[114, 256, 32, 112, 256, 32, 104, 256,
```

"tonight?" vs. "tonight"

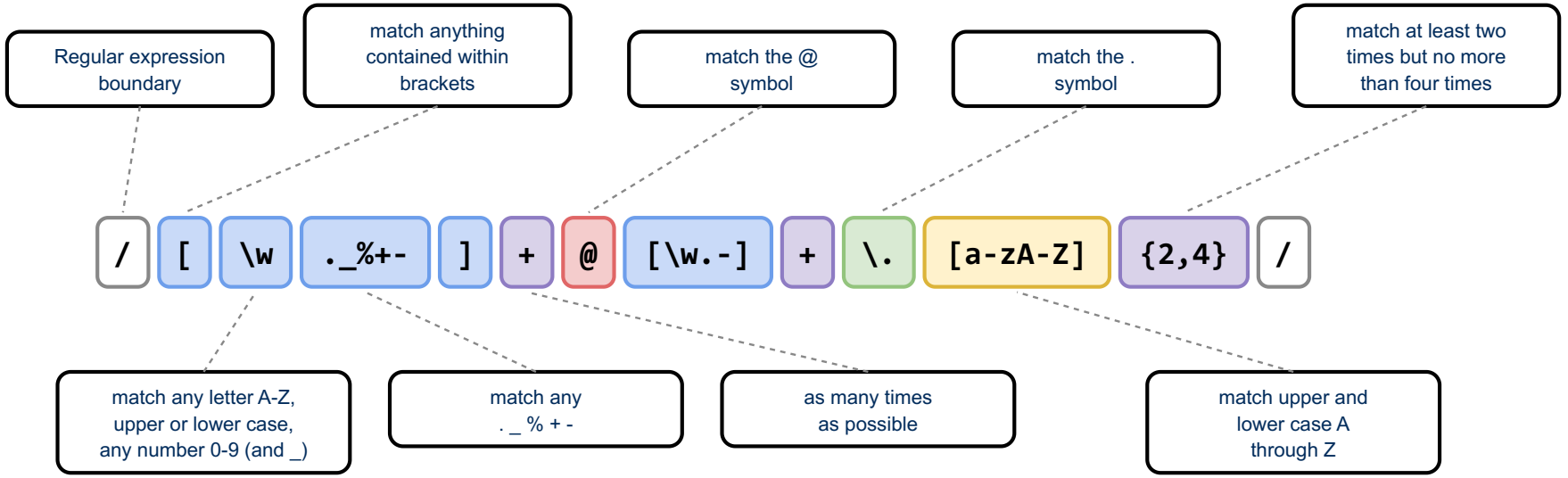
This is a simple algorithm, but it seems very silly that **"tonight?"** is a different token than **"tonight"**.



BPE is excellent at breaking down words statistically, but **it can make suboptimal merges** like combining punctuation with words.

`programming`: Regex

Regex allows us to define patterns that we can use to filter `strings`.



NOTE: Regex is super confusing and almost a language unto itself, **only sickos have it memorized**. **Just use a cheat sheet or ask chat.**

NOTE: regex is short for **"regular expression"** (or "rational expression") and I actually met one of the creators while interviewing at Princeton!

Regex in Tokenization: GPT4 Tokenizer

What we can do is pre-split our text based on linguistic "landmarks" like whitespace and punctuation using **regex**.

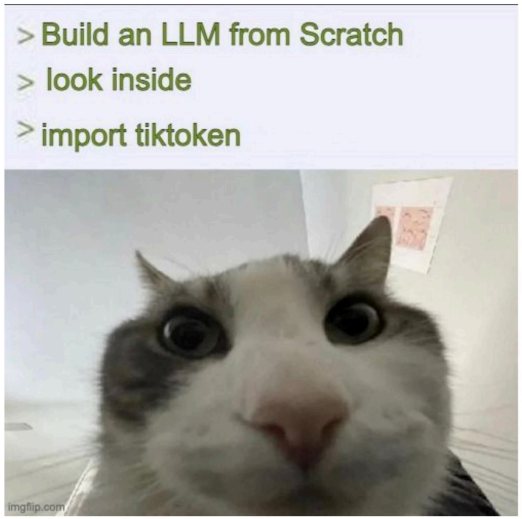
```
import regex
GPT4 = (r"'(?i:[sdmt]|ll|ve|re)|[^\r\n\p{L}\p{N}]?+"
        r"\p{L}+|\p{N}{1,3}| ?[^\s\p{L}\p{N}]++[\r\n]*"
        r"|\s*[\r\n]|\s+(?!\S)|\s+")
text = "No cap are u rolling the party tonight?"
print(regex.findall(GPT4, text))
```

OUTPUT

```
['No', ' cap', ' are', ' u', ' rolling'
```

regex acts as a "pre-filter" to prevent these issues, ensuring that letters, numbers, punctuation, and whitespace split.

A detailed discussion and implementation of BPE is out of the scope of this book, but in short, it builds its vocabulary by iteratively merging frequent characters into sub-



Tokenization vs. Meaning

`"dirty clean"`



`"drty"`



Asked "What is the opposite of drty", ChatGPT answers:

```
The opposite of "drty" (presumably "dirty") is clean.
```

The token ids for **"dirty"** and **"drty"** have nothing in common, but the model still works out what we meant.